TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Smart City 2.0 — AI-Powered · Full Curriculum

2-day camp (5 hrs/day) · 12–15 yrs · Advanced

Full facilitator curriculum **PREMIUM**

Overview

The ultimate crossover: build an Arduino smart city, then give it an AI brain.

DETAIL	
Track	Combo · Arduino + AI
Format	2-day camp (5 hrs/day)
Ages	12–15 yrs
Level	Advanced
Price	from KES 8,000
Standalone	No
Prerequisites	Any 2 free Arduino activities, Prompting power (free)

Course Overview

Smart City 2.0 — AI-Powered is TinkerWithMe's flagship crossover camp: it takes everything students know about physical computing and fuses it with applied AI. Over two five-hour days, teams of three to four build a fully working, multi-sensor "city district" on an Arduino Uno — a traffic light, an automatic parking gate, a light-sensing street lamp, and an environment station — then, on day two, they teach that city to think. A webcam and Google's Teachable Machine give the intersection a "vision layer" that classifies traffic density in real time and adjusts signal timing live over a serial link, while ChatGPT turns the raw sensor log into a written city-planning report, exactly the kind of document a real municipal engineer would read.

The arc of the course mirrors real embedded-AI product development: Day 1 is pure systems engineering — wiring, libraries, non-blocking code, calibration, and

physical build quality. Day 2 is the "add intelligence" layer that most Arduino camps never reach — training a computer-vision model, bridging a browser AI prediction to a microcontroller over USB, and using a large language model to interpret data the way a human analyst would. Every team spends the two days as a small engineering firm: assigning roles, debugging together, and rehearsing a client pitch.

Students leave with two physical/digital deliverables: their working Arduino city module (mounted on its cardboard diorama base, wired and coded) and a printed AI-generated city-planning report bearing their own sensor data and their own trained vision model's classifications. They also leave having presented both, live, to an audience of parents — the full sense-think-act-communicate loop of a real smart city.

Learning Outcomes

- Design and wire a multi-sensor embedded system on Arduino Uno, combining digital outputs, PWM servo control, ultrasonic ranging, an analog voltage-divider sensor, and a digital humidity/temperature module on a single board.
- Write non-blocking Arduino code using `millis()`-based state machines so several subsystems (traffic light, parking gate, street lamp, environment logger) run concurrently without `delay()` freezing the board.
- Train, test, and export an image-classification model in Teachable Machine, understanding classes, training samples, and confidence thresholds.
- Bridge a browser-based AI prediction to physical hardware using the Web Serial API, closing the loop from camera to classification to actuator.
- Read and interpret a live serial data stream, and craft effective prompts that turn raw sensor logs into a structured, human-readable report using ChatGPT.
- Collaborate inside an engineering team with divided roles (wiring lead, code lead, AI lead, presenter) to integrate hardware, software, and AI into one working demo under time pressure.
- Communicate a technical build to a non-technical audience through a live, rehearsed demonstration and a written report.

Materials Master List

ITEM	SPEC	QTY PER TEAM	NOTES
Arduino Uno R3 (or Elegoo/compatible)	ATmega328P, USB-B port	1	One board stays with the team for both days
Breadboard	830 tie-point, half+ size	1	
USB cable	Type A–B	1	Also used for Web Serial on Day 2 — must be data-capable, not charge-only
Jumper wire pack	M-M, M-F, F-F, 40 pcs each	1 pack	
LED, red 5mm	Standard diffused	2	Traffic-red + parking-occupied
LED, yellow 5mm	Standard diffused	1	Traffic-amber
LED, green 5mm	Standard diffused	2	Traffic-green + parking-free
LED, white or warm-yellow 5mm	Standard diffused	1	Street lamp
Resistor 220Ω	1/4 W	6	One per LED
Resistor 10kΩ	1/4 W	1	LDR voltage divider
LDR (photoresistor)	5mm GL5528 or similar	1	
Ultrasonic distance sensor	HC-SR04	1	Parking bay occupancy
Micro servo motor	SG90, 180°	1	Parking boom gate
Temperature & humidity sensor	DHT11 module (3-pin, breakout board with onboard resistor)	1	Environment station
Pushbutton (spare/ backup)	12mm tactile	1	For extension challenges only
Cardboard sheet	A2 size, 3mm thick	1	City diorama base
Craft supplies	Coloured paper, small boxes, glue gun + sticks, scissors, markers, masking tape	1 set	Buildings and roads on the diorama
Toy die-cast cars	Small scale, assorted colours	15–20	Used to physically stage "low/ medium/high" traffic for Teachable Machine training

Printed road/city grid template	A2, pre-drawn intersection + parking bay	1	Facilitator-provided handout
Laptop	Arduino IDE 2.x installed, Chrome or Edge browser	1	Chrome/Edge required for Web Serial API on Day 2
Webcam	Built-in laptop cam or USB webcam	1	
Internet / Wi-Fi access	≥5 Mbps per laptop	Venue-wide	For Teachable Machine and ChatGPT
9V battery + barrel clip	Standard 9V with DC plug adapter	1	Optional standalone power for showcase demo
Power strip / extension cable	4–6 outlet	1 per 2 teams	
Multimeter	Basic digital	1 per 2 teams (shared)	Debugging continuity/voltage
Safety glasses	Standard clear	1 per student	Glue gun use
Printed worksheets	Wiring diagram, data-log sheet, AI prompt sheet	1 set per student	Facilitator-provided

Session Plans

Day 1 — Build: Assembling the Arduino Smart-City Module

Objectives

- Wire and code four subsystems — traffic light, parking gate, street lamp, environment station — onto one Arduino Uno.
- Use a `millis()`-based state machine so all subsystems run concurrently without blocking one another.
- Mount the working electronics onto a physical city diorama ready for AI integration tomorrow.

Timing (300 minutes)

- 00:00–00:15 — Welcome, recap of prior Arduino skills, safety briefing (glue gun, sensor handling)
- 00:15–00:35 — City planning brainstorm and team role assignment

- 00:35–01:05 — Traffic light circuit wiring + basic blink test
- 01:05–01:45 — Traffic light `millis()` state-machine code
- 01:45–02:00 — BREAK
- 02:00–02:30 — Parking module wiring (ultrasonic sensor + servo gate)
- 02:30–03:00 — Parking code + gate behaviour test
- 03:00–03:20 — Street lamp (LDR) wiring + code
- 03:20–03:50 — Environment sensor (DHT11) wiring + code, library install
- 03:50–04:20 — Full integration into one sketch + threshold calibration
- 04:20–04:35 — BREAK
- 04:35–04:55 — City diorama assembly — mount components onto cardboard base
- 04:55–05:00 — Wrap-up, save code, tidy stations

Step-by-step

1. Open with a 5-minute discussion: "What makes a city 'smart'? What would sensors need to measure?" Write student answers on the board and map them to the four subsystems they will build.
2. Assign roles within each 3–4 student team: Wiring Lead, Code Lead, Calibration/Test Lead, and (for teams of 4) Diorama Lead. Roles rotate responsibilities during integration.
3. Hand out the printed wiring diagram and the city grid template. Distribute component kits — do a parts-count check together before wiring starts.
4. Guide teams through wiring the three traffic LEDs first (lowest risk, quick win). Have every team run a simple blink test sketch before moving to the state machine, to confirm all three LEDs light correctly.
5. Introduce the concept of a state machine on the board: three states (RED, GREEN, YELLOW), each with a timer, transitioning in a fixed cycle. Explain why `delay()` would break the rest of the city ("if the light is asleep for 5 seconds, the parking sensor is asleep too"). Live-type the state machine function together, then have teams merge it into their own sketch.
6. After the break, introduce the ultrasonic sensor: demonstrate the trig/echo pulse concept with a simple "shout and count the echo" analogy. Wire it, test raw distance readings on the Serial Monitor before touching the servo.

7. Introduce the servo as the parking boom gate. Attach it, sweep it manually with `gateServo.write()` commands typed live, then wire the logic so the gate opens when the bay is empty and closes when a car (or a hand, or a toy car) is within 10 cm.
8. Wire the LDR as a voltage divider — draw the divider circuit on the board and explain why a resistor pairs with the LDR to produce a variable voltage. Read raw analog values in the Serial Monitor in bright vs covered conditions to help teams pick their own `darkThreshold`.
9. Introduce the DHT11: install the "DHT sensor library" by Adafruit and its dependency "Adafruit Unified Sensor" via Library Manager together as a whole group (Sketch → Include Library → Manage Libraries). Wire it, print temperature and humidity to Serial.
10. Integration: teams now combine all four blocks into the single master sketch (provided below). Walk around and help each team calibrate `darkThreshold` and `carDistanceThreshold` to their classroom's actual lighting and sensor placement.
11. Diorama time: teams glue/tape their breadboard and Arduino onto the cardboard base, position the traffic LEDs at an "intersection," the ultrasonic sensor facing a drawn parking bay, the LDR near a paper "street lamp" pole, and the DHT11 in an open "park" area. Keep wires long enough that nothing is under tension.
12. Close the day: each team saves their sketch (File → Save As "Day1_CityModule"), and the facilitator confirms every board runs the full integrated demo before students leave.

Build detail

Wiring list (component → Arduino pin):

COMPONENT	CONNECTION
Traffic LED — Red	Anode → D2 through 220Ω resistor; cathode → GND
Traffic LED — Yellow	Anode → D3 through 220Ω resistor; cathode → GND
Traffic LED — Green	Anode → D4 through 220Ω resistor; cathode → GND
Parking LED — Occupied (red)	Anode → D5 through 220Ω resistor; cathode → GND
Parking LED — Free (green)	Anode → D8 through 220Ω resistor; cathode → GND
HC-SR04 — Trig	D7
HC-SR04 — Echo	D6

HC-SR04 — VCC / GND	5V / GND
Servo (gate) — Signal	D9
Servo — VCC / GND	5V / GND
Street lamp LED	Anode → D10 through 220Ω resistor; cathode → GND
DHT11 — Data	D11 (module has onboard pull-up resistor)
DHT11 — VCC / GND	5V / GND
LDR	One leg → 5V; other leg → A0 *and* → 10kΩ resistor → GND (voltage divider)

Complete Arduino code (Day 1 master sketch):

```
#include <Servo.h>
#include <DHT.h>

// ---------- Pin map ----------
const int redPin           = 2;
const int yellowPin        = 3;
const int greenPin         = 4;
const int parkOccupiedLED  = 5;
const int echoPin          = 6;
const int trigPin          = 7;
const int parkFreeLED      = 8;
const int servoPin         = 9;
const int streetLightPin   = 10;
const int dhtPin           = 11;
const int ldrPin           = A0;

#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
Servo gateServo;

// ---------- Traffic light state machine ----------
enum TrafficState { RED, GREEN, YELLOW };
TrafficState currentState = RED;
unsigned long stateStartTime = 0;

unsigned long redTime    = 5000;
unsigned long greenTime  = 5000;
unsigned long yellowTime = 2000;

// ---------- Parking ----------
const int carDistanceThreshold = 10; // cm - closer than this = slot occupied
bool slotOccupied = false;

// ---------- Street light ----------
const int darkThreshold = 500; // adjust after testing with classroom light

// ---------- Timers for non-blocking sensor reads ----------
unsigned long lastParkingCheck = 0;
unsigned long lastEnvCheck = 0;
unsigned long lastPrintTime = 0;
```

```
// ---------- Environment ----------
float lastTemp = 0;
float lastHum = 0;

void setup() {
  Serial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(parkOccupiedLED, OUTPUT);
  pinMode(parkFreeLED, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(streetLightPin, OUTPUT);

  gateServo.attach(servoPin);
  gateServo.write(90); // start with gate open (slot free)

  dht.begin();

  currentState = RED;
  stateStartTime = millis();
  digitalWrite(redPin, HIGH);

  Serial.println("Smart City module booting...");
}

void loop() {
  updateTrafficLight();
  checkParking();
  checkStreetLight();
  checkEnvironment();
  logStatus();
}

// ---------- Traffic light ----------
void updateTrafficLight() {
  unsigned long elapsed = millis() - stateStartTime;

  switch (currentState) {
    case RED:
      if (elapsed >= redTime) {
        currentState = GREEN;
        stateStartTime = millis();
        digitalWrite(redPin, LOW);
        digitalWrite(greenPin, HIGH);
      }
      break;
    case GREEN:
      if (elapsed >= greenTime) {
        currentState = YELLOW;
        stateStartTime = millis();
        digitalWrite(greenPin, LOW);
        digitalWrite(yellowPin, HIGH);
      }
      break;
    case YELLOW:
      if (elapsed >= yellowTime) {
```

```
        currentState = RED;
        stateStartTime = millis();
        digitalWrite(yellowPin, LOW);
        digitalWrite(redPin, HIGH);
      }
      break;
  }
}

// ---------- Parking ----------
long readDistanceCM() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  long distance = duration * 0.034 / 2;
  if (distance == 0) distance = 400; // no echo = treat as far/empty
  return distance;
}

void checkParking() {
  if (millis() - lastParkingCheck < 300) return; // sample every 300ms
  lastParkingCheck = millis();

  long distance = readDistanceCM();
  slotOccupied = (distance > 0 && distance < carDistanceThreshold);

  if (slotOccupied) {
    digitalWrite(parkOccupiedLED, HIGH);
    digitalWrite(parkFreeLED, LOW);
    gateServo.write(0);   // gate closed
  } else {
    digitalWrite(parkOccupiedLED, LOW);
    digitalWrite(parkFreeLED, HIGH);
    gateServo.write(90);  // gate open
  }
}

// ---------- Street light ----------
void checkStreetLight() {
  int lightLevel = analogRead(ldrPin);
  if (lightLevel < darkThreshold) {
    digitalWrite(streetLightPin, HIGH);
  } else {
    digitalWrite(streetLightPin, LOW);
  }
}

// ---------- Environment ----------
void checkEnvironment() {
  if (millis() - lastEnvCheck < 2000) return; // DHT11 needs ~2s between reads
  lastEnvCheck = millis();
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (!isnan(h) && !isnan(t)) {
    lastHum = h;
    lastTemp = t;
  }
```

```
}

// ---------- Serial data log for tomorrow's AI report ----------
void logStatus() {
  if (millis() - lastPrintTime < 1000) return;
  lastPrintTime = millis();

  String stateName = (currentState == RED) ? "RED" : (currentState == GREEN) ? "GREEN" : "YELLOW";

  Serial.print("TRAFFIC:");
  Serial.print(stateName);
  Serial.print(",PARKING:");
  Serial.print(slotOccupied ? "OCCUPIED" : "FREE");
  Serial.print(",LDR:");
  Serial.print(analogRead(ldrPin));
  Serial.print(",TEMP:");
  Serial.print(lastTemp);
  Serial.print(",HUM:");
  Serial.println(lastHum);
}
```

This full sketch is the "body" the AI will control tomorrow: the `greenTime` variable and the Serial log format above are intentionally left easy to hook into — Day 2 reads that same log and writes to that same variable.

Checks for understanding

- Why does the state machine use `millis()` instead of `delay()`? What would break if we used `delay()` here?
- What does the LDR voltage divider actually measure, and why do we need the 10kΩ resistor?
- If the ultrasonic sensor reads 0, what does our code assume, and why is that a safe default for a parking bay?

Common mistakes & fixes

MISTAKE	FIX
LED wired backwards (doesn't light)	Check LED polarity — longer leg (anode) goes toward the resistor/pin, flat side/short leg to GND
Servo jitters or resets the board	Too much current draw from 5V pin — move servo VCC to the 9V battery pack or a separate USB power bank for the final demo
DHT11 returns "nan"	Check the 2-second minimum read interval is respected; reseat the 3-pin connector; confirm library installed correctly
Ultrasonic sensor gives wildly inconsistent readings	Make sure nothing is closer than 2cm (sensor's minimum range) and the sensor is facing squarely at the target, not at an angle

Traffic lights never change state	Check that `stateStartTime` is being reset inside every `if` branch, not just declared once in `setup()`

Day 2 — Add Intelligence: Vision, Reports & the Parent Showcase

Objectives

- Train a Teachable Machine image classifier that reads traffic density from a webcam and bridge its prediction to the Arduino over Web Serial.
- Use ChatGPT to transform the module's live sensor log into a written city-planning report.
- Present the finished AI-enhanced smart city to parents in a rehearsed, confident demo.

Timing (300 minutes)

- 00:00–00:15 — Recap Day 1, introduce "giving the city an AI brain"
- 00:15–00:45 — Teachable Machine walkthrough: accounts, project types, image classification basics
- 00:45–01:30 — Capture training images (Low/Medium/High traffic classes), train the model
- 01:30–01:45 — BREAK
- 01:45–02:15 — Test and refine the model; fix confused classes; get the shareable model link
- 02:15–02:45 — Build the browser page: paste model link, wire the Connect/Start buttons, confirm live predictions on screen
- 02:45–03:15 — Connect Web Serial to Arduino: update the sketch, test camera → classification → green-light-timing change end to end
- 03:15–03:30 — BREAK
- 03:30–04:00 — ChatGPT city-planning report: gather log data, craft prompt, generate and edit report
- 04:00–04:30 — Rehearse presentations, assign speaking roles, prepare demo script
- 04:30–05:00 — Parent showcase — live demos and Q&A

Step-by-step

1. Recap: reconnect every team's Arduino, confirm yesterday's sketch still runs, and reset the diorama on the table. Frame the day: "Today your city stops running on fixed timers and starts making decisions."
2. Introduce Teachable Machine (teachablemachine.withgoogle.com) on the projector. Explain image classification: the model looks at a picture and sorts it into one of the classes we teach it, with a confidence score.
3. Each team creates a new "Standard image project" with exactly three classes: *Low Traffic*, *Medium Traffic*, *High Traffic*. Using their toy cars on the diorama's drawn road, teams capture roughly 40–60 webcam images per class: Low = 1–2 cars visible, Medium = 4–6 cars, High = 8+ cars clustered at the intersection. Vary angle and lighting slightly for each capture batch.
4. Click "Train Model" together, watch the training graph, then test live in the "Preview" pane by holding up different car arrangements. If a class is consistently misclassified, add 15–20 more images for that class and retrain.
5. Once happy, teams click "Export Model" → "Upload (shareable link)" and copy the generated URL — this is pasted into the webpage template in the next step.
6. Distribute the HTML/JS template below. Explain each part on the board: `tmImage.load()` loads the trained model, the webcam loop calls `predict()` continuously, and the highest-probability class is mapped to a single character (L, M, or H) sent to the Arduino.
7. Critical setup step: the Arduino IDE's Serial Monitor **must be closed** before opening the webpage, otherwise the serial port is locked and Web Serial's "Connect to Arduino" button will fail. Teach this as a rule, not a suggestion.
8. Teams open the HTML file in Chrome or Edge (double-click, or use a simple local server if the browser blocks camera access from a file:// path), click "Connect to Arduino," select their board's serial port, then click "Start Camera." Hold up toy-car arrangements and watch the predicted class change, and confirm the Arduino's Serial Monitor (once safely reopened after testing) prints "AI-COMMAND" lines and the green light duration visibly changes.
9. Update the Arduino sketch with the `checkAICommand()` addition (full sketch below) and re-upload. Test the whole loop live: fewer toy cars → short green; a cluster of cars → long green.
10. After the break, move to ChatGPT. Open the Arduino Serial Monitor, let the city run for 60–90 seconds, and have students copy 15–20 lines of the log. Walk

through the example prompt together on the projector, then each team pastes their own data and generates their own report.

11. Teams review the AI-generated report as a group, correcting anything factually odd, and format it into a one-page printable or a slide for the showcase.
12. Rehearsal: assign each team member one part of the live demo (wiring/build explanation, AI vision demo, report readout) and run through it once, timed to 3 minutes per team.
13. Showcase: parents circulate; each team runs their full demo — physical city, live camera classification changing the light timing, and their printed AI report — followed by 2 minutes of Q&A per team.

Build detail

AI tools and workflow:

- **Tool 1 — Teachable Machine** (teachablemachine.withgoogle.com, free, no login required for basic use): trains an image classifier in-browser using transfer learning, exportable as a shareable model link.
- **Tool 2 — A plain HTML/JS webpage** (provided below) that loads the exported model, runs the webcam, and uses the **Web Serial API** (built into Chrome/Edge) to send single-character commands to the Arduino over the same USB cable used for programming.
- **Tool 3 — ChatGPT** (or the classroom's provided AI chat account): takes the pasted sensor log and produces a structured written report.

Complete webpage code (save as `index.html`, paste the team's own Teachable Machine model link where marked):

```
<!DOCTYPE html>
<html>
<head>
<title>Smart City AI Traffic Vision</title>
</head>
<body>
<h1>Smart City Traffic Density Classifier</h1>
<button id="connectBtn">Connect to Arduino</button>
<button id="startBtn">Start Camera</button>
<div id="webcam-container"></div>
<div id="label-container"></div>

<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@1.3.1/dist/tf.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@teachablemachine/image@0.8/dist/teachablemachine-
image.min.js"></script>
<script>
const URL = "PASTE_YOUR_MODEL_LINK_HERE/"; // must end in a forward slash
```

```
let model, webcam, labelContainer, maxPredictions;
let port, writer;

async function connectSerial() {
  port = await navigator.serial.requestPort();
  await port.open({ baudRate: 9600 });
  writer = port.writable.getWriter();
  alert("Arduino connected!");
}

async function initCamera() {
  const modelURL = URL + "model.json";
  const metadataURL = URL + "metadata.json";

  model = await tmImage.load(modelURL, metadataURL);
  maxPredictions = model.getTotalClasses();
```